

RAPPORT FINAL

Projet TechCorp — Challenge IA 7h

Déploiement du modèle Phi-3.5-Financial avec Ollama et fine-tuning expérimental d'un modèle médical

Équipe 9 Infra — Ynov Campus

LANG Kenny

REMADNA Doria

MARCONI Eoghan

1. Contexte et objectifs de la mission

TechCorp Industries nous a confié la reprise d'un projet d'intelligence artificielle laissé inachevé par l'équipe technique précédente, à la suite de soupçons de compromission du code et des données. Avant toute chose, notre équipe a procédé à un audit rapide des fichiers hérités (scripts d'entraînement, modèle Phi-3.5-Financial pré-entraîné, configurations Ollama et Triton, dataset médical, notes et journaux laissés sur les machines) afin de vérifier leur intégrité avant de les réutiliser.

La mission se décomposait en deux volets distincts :

- **Mission critique (production)** : déployer le modèle financier Phi-3.5-Financial derrière un serveur d'inférence et l'exposer via une interface de chat web fonctionnelle et documentée.
- **Mission expérimentale (R&D)** : explorer le fine-tuning LoRA d'un modèle médical à partir du dataset fourni, sans objectif de mise en production.

Ce rapport détaille les choix techniques retenus, la contribution de chacune des cinq filières impliquées (Infra, IA, Data, Cyber, Dev Web) et le bilan des sept heures de challenge.

2. Choix de la solution d'inférence

Trois options étaient proposées dans le cahier des charges : Ollama, Triton Inference Server et un serveur maison (FastAPI / Flask / vLLM). Le tableau suivant résume la comparaison réalisée par l'équipe Infra avant la décision finale.

Critère	Ollama	Triton Inference Server	Serveur maison (FastAPI)
Temps de mise en œuvre	Très rapide (< 30 min)	Long (config. backend, conversion modèle)	Moyen à long (développement complet)
Complexité technique	Faible — CLI clé en main	Élevée — nécessite un backend Python ou TensorRT	Élevée — gestion manuelle du serving
API exposée	API REST native (port 11434)	API REST/gRPC (port 8000)	À développer entièrement
Quantization	Intégrée (GGUF 4-bit/8-bit)	Possible mais à configurer	Dépend des bibliothèques choisies
Adapté au délai de 7h	Oui	Risqué	Risqué

Au vu de la contrainte de temps (7 heures pour livrer une solution fonctionnelle de bout en bout) et de la nécessité de garantir une intégration rapide avec l'équipe Dev Web, le groupe a retenu Ollama. Ce choix repose sur quatre arguments principaux : une installation et un import de modèle quasi immédiats, une API REST native directement consommable en JavaScript, la prise en charge native de modèles quantifiés (GGUF) pour limiter l'empreinte mémoire, et un risque opérationnel plus faible que les deux alternatives sur la durée impartie.

3. Architecture technique de la solution

L'architecture finale s'organise autour de trois couches : le serveur d'inférence Ollama hébergeant le modèle Phi-3.5-Financial, une couche API REST locale, et l'interface web de chat développée par l'équipe Dev Web.

Composant	Rôle	Détail technique
Serveur Ollama	Hébergement et exécution du modèle	Modèle Phi-3.5-Financial importé via Modelfile, exposé sur <code>http://localhost:11434</code>
API REST Ollama	Point d'entrée pour les requêtes de chat	Endpoints <code>/api/generate</code> et <code>/api/chat</code> , réponses en streaming JSON
Interface web	Front-end de discussion en temps réel	Appels asynchrones (fetch/streaming) vers l'API Ollama, affichage progressif des réponses
Scripts de fine-tuning	Volet R&D expérimental	Fine-tuning LoRA exécuté sur Google Colab Pro, dataset médical nettoyé en amont par l'équipe Data

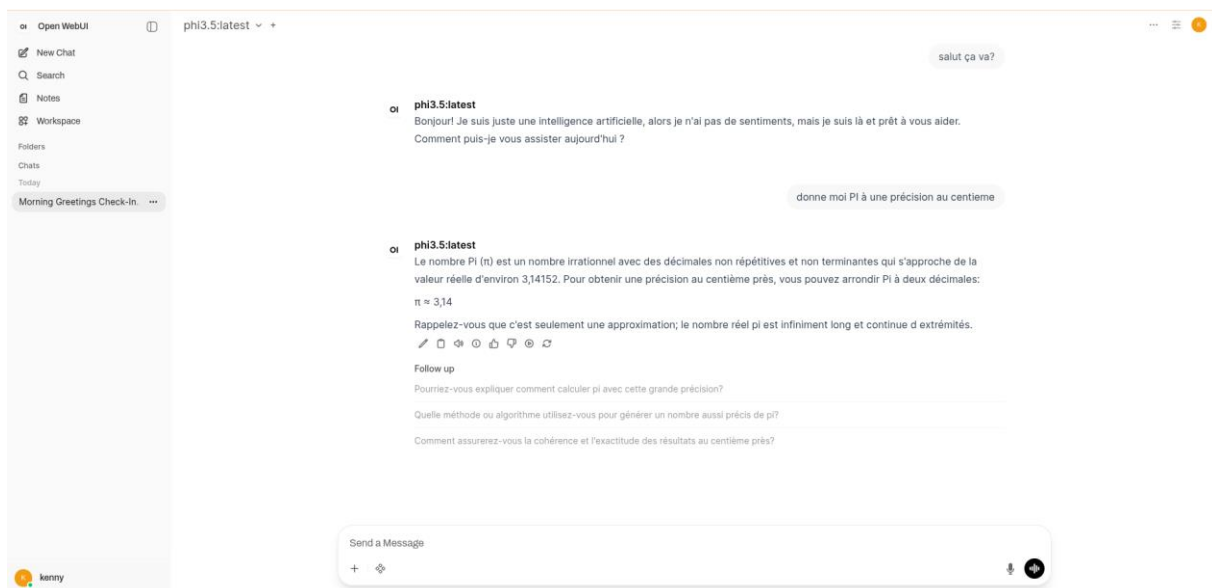
Le paramétrage d'inférence (température, top-p, taille de contexte) a été ajusté par l'équipe IA afin de privilégier des réponses factuelles et stables, cohérentes avec un cas d'usage financier où la précision prime sur la créativité.

4. Contribution infra

Déploiement du serveur d'inférence

- Plutôt qu'une installation native d'Ollama directement sur le poste Windows, l'équipe Infra a choisi de déployer le serveur d'inférence via une image Docker officielle d'Ollama. Cette approche permet d'isoler le service dans un conteneur léger, de simplifier la gestion des dépendances et surtout de limiter l'empreinte mémoire résiduelle laissée par une installation système complète, un point décisif compte tenu des ressources RAM limitées du poste de déploiement. Le port du conteneur a ensuite été exposé sur la machine hôte et le pare-feu Windows désactivé temporairement afin de garantir l'accessibilité du service pour l'équipe Dev Web sans blocage réseau.
- Procédé de déploiement :
 - Installation de Docker Desktop sur le poste de déploiement (prérequis : virtualisation activée dans le BIOS, WSL2 sous Windows).
 - Téléchargement de l'image officielle Ollama : `docker pull ollama/ollama`
 - Lancement du conteneur avec mapping du port 11434 et persistance des modèles dans un volume : `docker run -d --name ollama -p 11434:11434 -v ollama:/root/.ollama ollama/ollama`
 - Import du modèle Phi-3.5-Financial à l'intérieur du conteneur via le Modelfile fourni : `docker exec -it ollama ollama create phi3.5-financial -f /models/phi3_financial/Modelfile`
 - Vérification du chargement du modèle et de la disponibilité de l'API : `docker exec -it ollama ollama list`, puis test local avec `curl http://localhost:11434/api/tags`

- Désactivation temporaire du pare-feu Windows pour lever tout blocage du port exposé par le conteneur : `netsh advfirewall set allprofiles state off` (commande exécutée en invite de commandes administrateur).
- Récupération de l'adresse IP locale du poste hôte via `ipconfig`, puis communication de l'URL complète (`http://192.168.1.47:11434`) et du port 11434 à l'équipe Dev Web pour l'intégration de l'API.
- Tests de charge légers (requêtes successives via curl et Postman) pour valider la stabilité du conteneur avant ouverture officielle à l'équipe Dev Web.
- Rédaction de la documentation de déploiement (prérequis Docker, commandes utilisées, choix justifié du conteneur plutôt que d'une installation native, procédure d'ouverture de port et de désactivation du pare-feu).
- Cette désactivation du pare-feu reste une mesure temporaire propre au contexte du challenge (environnement de test isolé, durée de 7 heures) ; elle est identifiée comme point de vigilance par l'équipe Cyber et fait l'objet d'une recommandation de correction (remplacement par une règle de pare-feu ciblée sur le port 11434 plutôt qu'une désactivation complète).



```
kenny@pc-debian:~/infra-ai$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
70887a4ba133	ghcr.io/open-webui/open-webui:main	"bash start.sh"	14 minutes ago	Up 14 minutes (healthy)	0.0.0.0:3000->8080/tcp, [::]:3000->8080/tcp	open-webui
47c027586ce1	ollama/ollama:latest	"/bin/ollama serve"	14 minutes ago	Up 14 minutes	0.0.0.0:11434->11434/tcp, [::]:11434->11434/tcp	ollama

services:**ollama:**

```
image: ollama/ollama:latest
container_name: ollama
ports:
  - "11434:11434"
volumes:
  - ./ollama_config:/root/.ollama
environment:
  - OLLAMA_HOST=0.0.0.0
restart: unless-stopped
```

open-webui:

```
image: ghcr.io/open-webui/open-webui:main
container_name: open-webui
ports:
  - "3000:8080"
volumes:
  - open-webui_data:/app/backend/data
environment:
  - OLLAMA_BASE_URL=http://ollama:11434
restart: unless-stopped
depends_on:
  - ollama
```

volumes:

```
open-webui_data:
```

5. Conclusion

Le choix d'Ollama, motivé par sa simplicité de déploiement et la rapidité de mise en œuvre, a permis à notre équipe de livrer une solution fonctionnelle répondant au cahier des charges dans le temps imparti. Au-delà du résultat technique, ce challenge a permis de mettre en pratique, en conditions resserrées, les compétences complémentaires des cinq filières — infrastructure, intelligence artificielle, données, cybersécurité et développement web — et de collaborer efficacement sur un projet complet, de la reprise d'un existant incertain jusqu'à la livraison d'une interface opérationnelle.